

INSTITUCIÓN UNIVERSITARIA DE COLOMBIA

SESIÓN 2 — TENSOR DE COLOR Y ANÁLISIS ESTADÍSTICO

I. DATOS GENERALES DE LA ASIGNATURA

Asignatura:	Diseño de Aplicaciones para Dispositivos Móviles
Período Académico:	IV Semestre
Créditos:	3 créditos
Duración en Horas:	40 horas
Docente:	Amaury Giovanni Méndez Aguirre

CONTEXTO: LA FÍSICA DEL COLOR Y SU DIGITALIZACIÓN

En la clase anterior comprendimos que una imagen es una matriz. Hoy expandiremos esa idea: las imágenes a color son **Tensores tridimensionales**. Físicamente, los sensores de las cámaras capturan la luz separándola en tres longitudes de onda principales mediante un filtro de Bayer: Rojo, Verde y Azul (RGB).

Para la Inteligencia Artificial, entender el color no es un tema estético, sino de separación de características (Feature Extraction). Un algoritmo diseñado para detectar venas en la piel o enfermedades en cultivos agrícolas dependerá de aislar y transformar matemáticamente combinaciones específicas de estos tres canales antes de pasar la información a la red neuronal.

1. Descomposición del Tensor y Slicing Avanzado

Un tensor de imagen RGB en Python tiene la forma (*Alto, Ancho, Canales*). Para acceder a un canal individual sin usar bucles `for` (los cuales son ineficientes computacionalmente), utilizamos la notación de **Slicing (Rebanado)** de NumPy.

La sintaxis `tensor[:, :, 0]` le indica al intérprete: "Toma todas las filas, toma todas las columnas, pero extrae únicamente la profundidad 0".

Nota técnica crucial: Por razones de hardware heredadas de los años 90, la librería OpenCV carga los canales en orden inverso: **BGR (Azul = 0, Verde = 1, Rojo = 2)**.

```
import cv2
import numpy as np

# Supongamos un tensor de imagen cargado
imagen = cv2.imread('muestra.jpg')

# Extracción de matrices 2D individuales (Slicing)
canal_azul = imagen[:, :, 0]
canal_verde = imagen[:, :, 1]
canal_rojo = imagen[:, :, 2]

# Si quisieramos "apagar" el canal verde y azul para dejar solo el rojo:
imagen_solo_rojo = np.copy(imagen)
imagen_solo_rojo[:, :, 0] = 0 # Azul a 0
imagen_solo_rojo[:, :, 1] = 0 # Verde a 0
```

TALLER ANALÍTICO 1: OPERACIONES CON TENSORES (20 MIN)

Discutan y resuelvan:

1. Si ejecutamos la instrucción `recorte = imagen[100:200, 300:400, 1]` sobre una imagen de 1920x1080. ¿Cuáles son las dimensiones exactas (shape) de la variable `recorte` resultante y qué información específica contiene?
2. ¿Por qué es computacionalmente más eficiente aislar un canal usando *Slicing* (ej. `img[:, :, 0]`) en lugar de crear dos ciclos `for` anidados para iterar sobre la altura y anchura? (Pista: piensen en la arquitectura de la memoria).

Espacio para análisis y justificación...

2. Conversión a Escala de Grises: Un Producto Punto Oculto

Convertir una imagen a escala de grises no es simplemente promediar los tres canales $(R+G+B)/3$. El ojo humano es mucho más sensible a la luz verde que a la roja o azul. Por tanto, la conversión real utilizada en Computer Vision es una combinación lineal ponderada:

$$Y = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$

Esta ecuación se aplica a cada vector de píxel. En álgebra lineal, esto equivale a realizar un **Producto Punto (Dot Product)** entre el tensor de la imagen y un vector de pesos $W = [0.114, 0.587, 0.299]$ (ajustado al orden BGR).

TALLER DE LABORATORIO 1: TRANSFORMACIÓN DE ESPACIOS (25 MIN)

Reto en Python:

1. Abran su IDE. Creen un píxel BGR de prueba completamente amarillo intenso:
`pixel = np.array([0, 255, 255]).`
2. Calculen matemáticamente su valor en escala de grises usando la fórmula ponderada superior (escribanlo usando operaciones básicas de NumPy, no usen funciones de OpenCV todavía).
3. Impriman el resultado. ¿Qué valor de gris (intensidad de 0 a 255) arroja un amarillo puro?
4. Ahora, usen la función optimizada de OpenCV: `img_gris = cv2.cvtColor(imagen, cv2.COLOR_BGR2GRAY)` sobre una imagen real descargada de internet para corroborar el proceso.

3. El Histograma: Estadística de la Imagen

En el preprocesamiento para IA, necesitamos saber si una imagen tiene buen contraste antes de dársela a una red neuronal. El **Histograma** es un gráfico de distribución de

frecuencias que nos muestra cuántos píxeles existen para cada uno de los 256 niveles de intensidad (del 0 al 255).

- Si los datos se agrupan a la izquierda (cerca del 0), la imagen está subexpuesta (oscura).
- Si están a la derecha (cerca del 255), está sobreexpuesta.
- Si utilizan todo el espectro, tiene un contraste óptimo (ideal para Machine Learning).

```
import cv2
import matplotlib.pyplot as plt

imagen = cv2.imread('foto.jpg', cv2.IMREAD_GRAYSCALE)

# cv2.calcHist(imágenes, canales, máscara, tamaño_bins, rangos)
histograma = cv2.calcHist([imagen], [0], None, [256], [0, 256])

# Graficar usando Matplotlib
plt.plot(histograma)
plt.title("Distribución de Intensidades")
plt.xlabel("Valor del Píxel (0-255)")
plt.ylabel("Frecuencia (Cantidad de píxeles)")
plt.show()
```

TALLER DE LABORATORIO 2: ANÁLISIS ESTADÍSTICO (30 MIN)

Misión Final:

1. Carguen una imagen RGB de su elección en Python.
2. Separen la imagen en sus 3 canales (B, G, R).
3. Calculen el histograma de **cada canal por separado**.
4. Utilicen matplotlib para graficar los tres histogramas superpuestos en la misma figura (usando colores azul, verde y rojo para las líneas respectivamente).
5. Concluyan analizando el gráfico: ¿Cuál es el color dominante en la iluminación general de su fotografía?